Day 60 – API Management (APIM) – Part 9: Basic Authentication in APIM

Module: SAP API Management – validating Basic Auth with Basic Auth + Key Value Map + Raise Fault policies

---

Checked against SAP's documentation – nothing needed correcting. Your key statement, that the Basic Auth *Decode* operation cannot validate credentials by itself, is exactly what SAP's policy reference says. Two practical tips are added in Sections 3 and 4 (encrypted KVMs, and the *createNew* setting) – these are extras, not mistakes in your notes.

1. The requirement (start here first)

Already seen in APIM: **OAuth** and **API Key** based authentication. Now: how do we do **Basic Authentication** in APIM?

Requirement: Postman → APIM → **Basic Authentication (username & password)** → APIM must **validate** the credentials. How?

The catch: the Basic Authentication policy's **Decode** option only **reads** the username and password out of the header. **In APIM we cannot validate with it alone** – SAP's own documentation states that this policy does not enforce basic authentication on a request to an API proxy; it only Base64 encodes or decodes credentials.

So what do we do? Combine three policies: **Basic Authentication (Decode)** to read what the caller sent, **Key Value Map Operations** to read the *correct* credentials, and **Raise Fault** to reject the call when they don't match.

2. Validation – step by step (demo)

Step 1 – Decode the incoming credentials

```
Policies -> Basic Authentication (use the Help for the default content)
```

Setting	Value
Operation	**Decode**
User ref	**current.username**
Password ref	**current.password**
Source	**request.header.Authorization** (where the caller's Basic Auth header arrives)

Result: the caller's username and password are now available in the flow variables **current.username** and **current.password**.

Step 2 – Create the Key Value Map holding the correct credentials

```
APIM -> Configure -> Key Value Maps -> Create
```

KVM name	Key	Value
credentials	username	TestUser

credentials	password	TestPassword (note the capital letters – values are case-sensitive)

Step 3 – Read the stored credentials with Key Value Map Operations

Policies → **Key Value Map Operations** → **mapIdentifier = credentials** → **Get operation, two times** – one for the **username** and one for the **password**. Each Get assigns its value to a variable, e.g. **sapapim.username** and **sapapim.password**.

Step 4 – Raise Fault if the incoming credentials are wrong

If the incoming username or password is wrong, we need to throw an error – for this we use the **Raise Fault** policy, with a condition that compares the decoded value with the stored value:

```
RaiseFault (username): Condition -> (current.username != sapapim.username)
RaiseFault (password): Condition -> (current.password != sapapim.password)
```

- **Status Code = 401** (Unauthorized).
- **Error payload:** use a standard error body (you searched for a sample on Google).
- **Content-Type = application/json.**

Policy order in the flow: Decode and the two KVM Get policies must all run **before** the Raise Fault policies (the order between Decode and the KVM Gets does not matter). Sequence used: KVM & Decode → RaiseFault (username) → RaiseFault (password). Attach them in the **Proxy Endpoint PreFlow – incoming request**, so a bad call is stopped before it reaches the backend.

Alternative (also used in SAP's community tutorial): a single Raise Fault policy with both checks joined by OR – (current.username != "<user>") OR (current.password != "<password>").

3. Practical tip – keep stored credentials encrypted

Since this KVM holds a password, create it with the **Encrypt Key Value Map** option ticked. For an encrypted map, the variable that receives the value must start with **private.** (for example **private.sapapim.password**), and the Raise Fault condition then refers to that same variable name.

4. Now the Encode operation – sending credentials onward

Encode is the opposite direction: build a Basic Auth header to send to the **target** (for example CPI) – **"Add Auth Details."**

Setting	Value
Operation	**Encode**
User ref	**sapapim.username** (from the KVM)
Password ref	**sapapim.password** (from the KVM)
AssignTo	**request.header.Authorization** with **createNew = "true"** (as demonstrated) – the Base64-encoded *Basic ...* value is written into the Authorization header

*Tip: SAP's documented examples use createNew="false" when the policy should **overwrite** an Authorization header that already exists on the request. If the caller's original Authorization header is still present, check in Debug/Trace that the target receives only the new value.*

Verifying on the CPI side

CPI flow → **Trace on** → copy the **Basic** value from the Authorization header → paste it into a **Base64 decoder** to verify the username:password that actually arrived. (Remember: Base64 is **encoding, not encryption** – anyone can decode it, which is why Basic Auth must always travel over HTTPS.)

5. OAuth with cache (recap)

OAuth → **Cache: Populate Cache** (store the token) and **Lookup Cache** (reuse it) – demo given in class (concept from Days 54–55).

Quick-recall summary

- The Basic Auth policy only **encodes/decodes** – it does **not** validate. Validation = **Decode + KVM Get + Raise Fault**.
- Decode: **Source = request.header.Authorization**, outputs **current.username / current.password**.
- KVM **credentials** holds the correct username/password; two **Get** operations read them into variables.
- Raise Fault: condition **current.x != stored x**, **401**, **application/json** error payload.
- Encode builds the outgoing Basic header for the target (**AssignTo request.header.Authorization**); verify in CPI Trace with a Base64 decoder.
- Store passwords in an **encrypted** KVM and use the **private.** variable prefix.